

Design and Implement a Mini Programming Language Compiler using Flex and Bison

Metropolitan University, Sylhet, Bangladesh

Department of Computer Science and Engineering

Compiler Construction Lab

Team Binary Builders

Member	Student ID	GitHub username
Altaf Hosen	231-115-108	altafth
Rakibul Islam Rafi	231-115-109	CodeByRafi

Repository: <https://github.com/altafth/CC-Lab-Project-Group09>

Project detail	Confirmed value
Section	Section: C
Batch	Batch: 58
Instructor	Khalid bin Selim, Lecturer, Metropolitan University, Sylhet
Submission	Submission Deadline: 31 July 2026
Presentation	Presentation Date: 2 August 2026

Table of Contents

1. Introduction
 2. Objectives
 3. Language Specification
 4. Compiler Architecture
 5. Lexer Design
 6. Parser Design
 7. Abstract Syntax Tree
 8. Semantic Analysis
 9. Symbol Table
 10. Intermediate Code Generation (TAC)
 11. Challenges
 12. Testing
 13. Team Contribution
 14. Conclusion
 15. References
- Appendix A — Sample Program Source
- Appendix B — Build System (Makefile)

1. Introduction

The Compiler Construction Laboratory course concludes with a comprehensive group project that integrates every major phase of a compiler front-end into a single coherent software system. Throughout the semester the individual phases — lexical analysis, syntax analysis, semantic analysis, and intermediate code generation — were studied largely in isolation. This project requires their complete integration, exactly as a real compiler front-end is structured.

This report documents the design and implementation of a mini programming language compiler developed by Team Binary Builders. The compiler is built using Flex (for the lexical analyzer), Bison (for the syntax analyzer), and C (for the remaining phases). The final deliverable is a working compiler front-end that accepts source programs written in the instructor-defined language, performs all required analyses, and produces Three-Address Code (TAC) as intermediate representation. Source code is maintained at <https://github.com/altafth/CC-Lab-Project-Group09>.

Evaluation of the project is based on five components: project implementation, documentation, live demonstration, group presentation, and individual viva. A working compiler with clear error messages is academically as valuable as one that successfully compiles valid programs; both aspects are thoroughly addressed in this work.

2. Objectives

The primary objectives of this project are:

- Design and implement a complete compiler front-end pipeline for the instructor-specified language using Flex and Bison.
- Integrate six required modules — Lexical Analyzer, Syntax Analyzer, Abstract Syntax Tree, Symbol Table, Semantic Analyzer, and Intermediate Code Generator — into a single working system.
- Demonstrate correct recognition of keywords, identifiers, constants, operators, delimiters, comments, and lexical errors.
- Define and implement an unambiguous Context-Free Grammar (CFG) and recover from syntax errors using Bison's error token.
- Construct a hierarchical Abstract Syntax Tree and support nested scoping with a proper symbol table.
- Detect and report the full set of required semantic errors (undeclared variables, redeclaration, scope violations, type mismatches, invalid assignments, and invalid expressions).
- Generate correct Three-Address Code for arithmetic, relational, and logical expressions as well as control-flow constructs (if, if-else, while) and print statements.
- Produce clear, human-readable diagnostic messages that include line numbers wherever possible.
- Document the complete design, testing strategy, and results in a professional project report.

3. Language Specification

All groups implement a compiler for the same fixed language defined in Section 5 of the Project Manual. This section presents the complete formal description used by Team Binary Builders.

3.1 Data Types

Type	Description
int	Signed integer
float	Floating-point number
bool	Boolean (true / false)

3.2 Statements Supported

- Variable declaration (with optional initialization): `int x;` or `int x = 5;`
- Assignment: `x = 5;`
- Arithmetic, relational, and logical expressions
- if statement and if-else statement
- while loop
- print statement
- Nested blocks `{ ... }` with proper lexical scoping

3.3 Operators

Category	Operators
Arithmetic	<code>+ - * / %</code>
Relational	<code>< > <= >= == !=</code>
Logical	<code>&& !</code>

3.4 Other Lexical Elements

- Braces `{ }`, parentheses `( )`, semicolons `;` (statement terminators)
- Identifiers: must start with a letter or underscore, followed by letters, digits, or underscores
- Integer literals (e.g., 42), floating-point literals (e.g., 3.14)
- Boolean literals: `true`, `false`
- Single-line comments `// ...` and block comments `/* ... */` — both discarded by the lexer

3.5 Formal Context-Free Grammar (CFG)

The following CFG is the authoritative grammar implemented in the Bison specification. It is unambiguous except for the classic dangling-else ambiguity, which is resolved by Bison's default shift action.

```
program      ::= stmt_list
stmt_list    ::= /* empty */
```

```

| stmt_list stmt

stmt      ::= decl_stmt
           | assign_stmt
           | if_stmt
           | while_stmt
           | print_stmt
           | block

type      ::= 'int' | 'float' | 'bool'

decl_stmt ::= type ID ';'
           | type ID '=' expr ';'

assign_stmt ::= ID '=' expr ';'

if_stmt    ::= 'if' '(' expr ')' block
           | 'if' '(' expr ')' block 'else' block

while_stmt ::= 'while' '(' expr ')' block

print_stmt ::= 'print' expr ';'

block      ::= '{' stmt_list '}'

expr       ::= expr '+' expr | expr '-' expr
           | expr '*' expr | expr '/' expr | expr '%' expr
           | expr '<' expr | expr '>' expr
           | expr '<=' expr | expr '>=' expr
           | expr '==' expr | expr '!=' expr
           | expr '&&' expr | expr '||' expr
           | '!' expr | '-' expr
           | '(' expr ')'
           | ID | INT_LIT | FLOAT_LIT | BOOL_LIT

```

3.6 Operator Precedence and Associativity

Precedence	Operators	Associativity
1 (lowest)		Left
2	&&	Left
3	! (unary)	Right
4	< > <= >= == !=	Non-associative
5	+ - (binary)	Left
6	* / %	Left
7 (highest)	- (unary / UMINUS)	Right

3.7 Sample Program

examples/sample.mc

```

// This is a comment
int a = 10;
float b = 20.5;
bool c = true;

/* Block Comment Test */
int d = a / 2;      // Division test
int e = a % 3;      // Modulo test

if (a != 0 && b > 10.0) {
    print a;
}

```

```
}  
  
while (!(c == false)) {  
    print b;  
    c = false;  
}
```

4. Compiler Architecture

The compiler implements a classic multi-phase front-end pipeline. Source code flows through four sequential stages; each stage either produces an intermediate data structure for the next stage or reports errors and aborts.

4.1 Pipeline Overview

The following figure shows the complete compilation pipeline from source code to Three-Address Code:

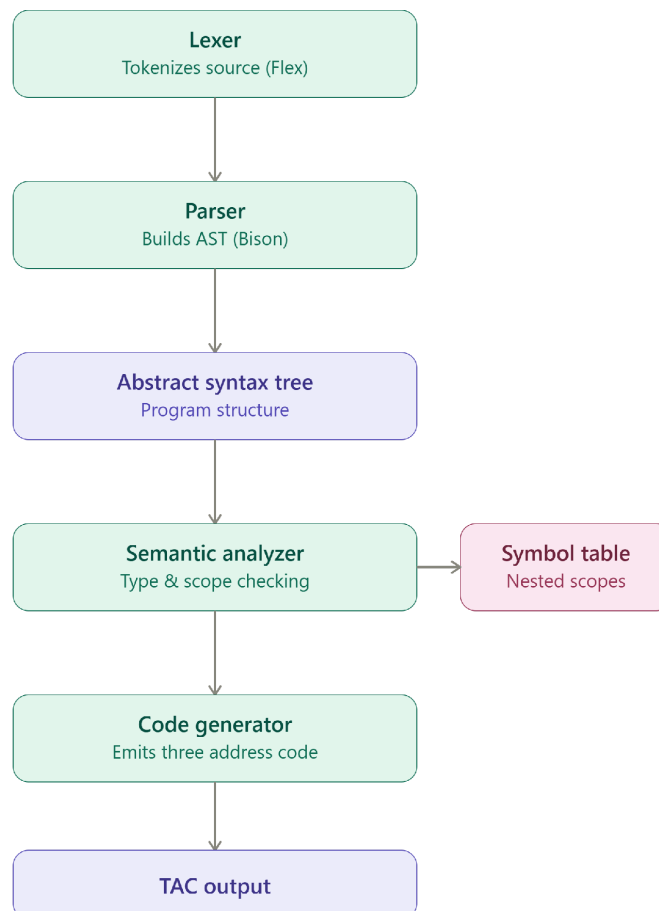


Figure 4.1: Compiler pipeline overview — Lexer → Parser → AST → Semantic Analyzer ↔ Symbol Table → Code Generator → TAC

Data flow between phases: Lexer produces a token stream consumed on-demand by the Parser; the Parser builds an AST; the Semantic Analyzer annotates the AST using the Symbol Table; the Code Generator walks the validated AST and emits TAC.

4.2 Module Responsibilities

Module	Directory	Responsibility
Lexer	src/lexer/	Tokenizes source; discards whitespace & comments; reports invalid characters.
Parser	src/parser/	Validates tokens against CFG; builds AST; recovers from syntax errors.
AST	src/ast/	Node hierarchy for every language construct; text-based tree printer.
Symbol Table	src/symbol_table/	Scope-chain with parent pointers; nested scope support.
Semantic	src/semantic/	Walks AST; checks declarations, types, scopes, expressions.
Code Generator	src/codegen/	Emits TAC; handles control flow with labels and jumps.
Driver	src/main.c	Orchestrates pipeline; supports --phase=selective output.

4.3 Two-Pass Compilation Strategy

The driver performs a dedicated lexical-only pass first (while (yylex() != 0)), reporting all lexical errors independently. If any lexical errors exist, compilation aborts. Otherwise the input is rewind and yyparse() is invoked, driving the lexer on-demand for the full parse. This design matches the pipeline diagram required by the Project Manual and gives each phase a distinct, independently reportable status.

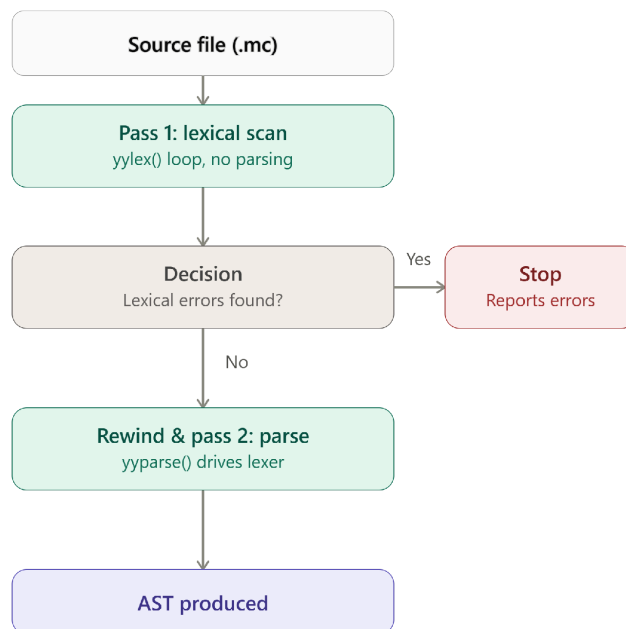


Figure 4.3: Two-pass compilation flow — lexical scan first; parse only if no lexical errors

4.4 Project Directory Structure

```
project-root/
├── docs/           # Architecture, grammar, design decisions, test summary
├── src/
│   ├── lexer/     # lexer.l (Flex specification)
│   ├── parser/    # parser.y (Bison grammar)
│   ├── ast/       # AST node definitions & printer
│   ├── semantic/  # Type checking, scope checking
│   ├── symbol_table/ # Scope-chain data structure
│   ├── codegen/   # TAC generator
│   └── main.c      # Pipeline driver
├── tests/         # Valid & invalid test programs
├── examples/      # Representative sample programs
├── build/         # Generated files (ignored by Git)
├── Makefile
└── README.md
```

5. Lexer Design

The lexical analyzer is generated by Flex from `src/lexer/lexer.l`. It reads the source file character-by-character and produces a stream of tokens for the parser.

5.1 Token Categories

Category	Examples / Pattern
Keywords	int, float, bool, if, else, while, print, true, false
Identifiers	[a-zA-Z][a-zA-Z0-9_]*
Integer literals	[0-9]+
Float literals	[0-9]+\.[0-9]+
Operators	+ - * / % < > <= >= == != && !
Delimiters	{ } () ;
Comments / Whitespace	Discarded (// and /* */ comments, spaces, tabs, newlines)

5.2 Design Decisions

- Keywords are defined as explicit patterns before the generic identifier rule, guaranteeing that Flex's longest-match rule never misclassifies a keyword as an identifier.
- Block comments are handled with a Flex exclusive start condition (`%x COMMENT`), correctly supporting multi-line comments and nested newline counting.
- Any character that does not match a valid token produces a diagnostic of the form "Lexical Error at line X: unexpected character c" and increments a global error counter. The driver aborts compilation if the counter is non-zero after the lexical pass.
- Line numbers are maintained by a global `line_num` variable that is incremented on every newline (including those inside comments).

6. Parser Design

The syntax analyzer is generated by Bison from `src/parser/parser.y`. It consumes the token stream produced by the lexer and builds an Abstract Syntax Tree while verifying that the input conforms to the CFG of Section 3.5.

6.1 Grammar Implementation

Every production of the formal CFG is realized as a Bison rule. Semantic actions construct AST nodes (`make_decl`, `make_assign`, `make_if`, `make_while`, `make_print`, `make_binop`, `make_unop`, etc.). Operator precedence and associativity are declared with `%left`, `%right` and `%nonassoc` directives, exactly matching the precedence table of Section 3.6.

6.2 Conflict Resolution

The only shift/reduce conflict is the classic dangling-else ambiguity. Bison's default resolution (prefer shift) binds an else clause to the nearest unmatched if, which is the conventional and expected behaviour for this language. No other conflicts remain after the precedence declarations.

6.3 Error Recovery

Basic error recovery is implemented with the production:

```
stmt : ... | error ';' { $$ = NULL; yyerrok; }
```

When a syntax error occurs, the parser skips tokens until a semicolon is found, then resumes. This allows multiple syntax errors to be reported in a single compilation run instead of aborting after the first error.

7. Abstract Syntax Tree

As parsing succeeds, the semantic actions of the Bison grammar construct an Abstract Syntax Tree. The AST removes purely syntactic noise (parentheses, semicolons) while retaining the hierarchical structure of the program.

7.1 Node Hierarchy

Each node is an instance of the `ASTNode` structure defined in `src/ast/ast.h`:

- `NODE_PROGRAM`, `NODE_BLOCK` — list containers
- `NODE_DECL` — variable declaration (optionally with initializer)
- `NODE_ASSIGN` — assignment statement
- `NODE_IF` — if / if-else (condition, then-block, optional else-block)
- `NODE_WHILE` — while loop (condition, body)
- `NODE_PRINT` — print statement
- `NODE_BINOP`, `NODE_UNOP` — binary and unary operators
- `NODE_ID`, `NODE_INT_LIT`, `NODE_FLOAT_LIT`, `NODE_BOOL_LIT` — leaves

Every node records the source line number on which it was created. After semantic analysis, expression nodes also carry an `eval_type` field (`TYPE_INT`, `TYPE_FLOAT`, `TYPE_BOOL`, or `TYPE_UNKNOWN`).

7.2 AST Visualization

A recursive text-based printer (`print_ast`) produces an indented tree representation of the AST. The figure below shows an example Abstract Syntax Tree for the assignment statement `c = a + b * 2`. Each node represents a language construct: Assign at the root, BinOp nodes for operators, and leaf nodes for identifiers and literals. Graphviz visualization is listed as an optional bonus feature and was not required for the core deliverable.

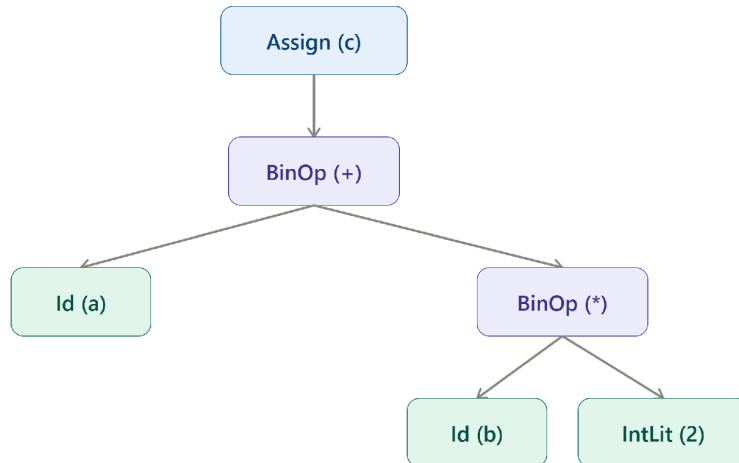


Figure 7.2: Example AST for `c = a + b * 2` — `Assign` → `BinOp(+)` → `Id(a)` and `BinOp(*)` → `Id(b)`, `IntLit(2)`

8. Semantic Analysis

The semantic analyzer (`src/semantic/semantic.c`) walks the AST after parsing and enforces rules that a context-free grammar alone cannot express. All detected errors are collected and reported together; the analyzer does not stop at the first error.

8.1 Errors Detected

Error Type	Example / Message
Undeclared variable	undeclared variable 'x'
Redeclaration	redeclaration of variable 'x'
Scope violation	undeclared / assignment to undeclared variable (outside declaring block)
Type mismatch (init)	cannot initialize 'bool' variable with 'float' expression
Type mismatch (assign)	cannot assign 'float' expression to 'bool' variable 'flag'
Invalid expressions	logical operators require bool operands; arithmetic/relational require numeric operands

8.2 Type Checking Strategy

Each expression node receives an `eval_type` during the semantic walk. Binary operators check that both operands have compatible types; logical operators require `TYPE_BOOL`; arithmetic and relational operators

require numeric types. Assignment and declaration-with-initializer compare the left-hand side type with the expression's `eval_type`. Conditions of `if` and `while` must evaluate to `bool`.

9. Symbol Table

The symbol table (`src/symbol_table/`) tracks every declared identifier together with its type, scope level, and declaration line number.

9.1 Data Structure

Scopes are represented as a parent-pointer linked list (scope chain). Entering a block pushes a new scope; exiting a block pops it. Lookup walks from the current scope toward the global scope, implementing classic lexical scoping. A variable declared inside an `if` or `while` block is therefore invisible outside that block.

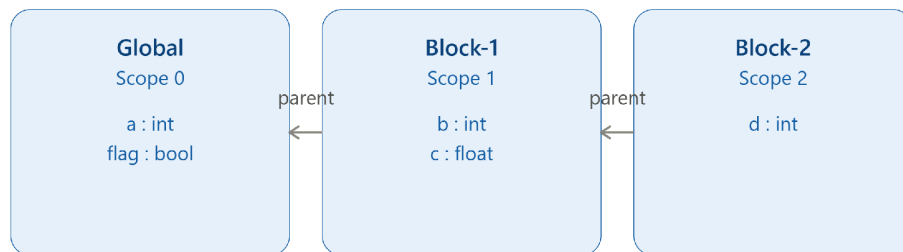


Figure 9.1: Symbol table scope chain — nested scopes linked by parent pointers (Global ← Block-1 ← Block-2)

9.2 Design Notes

- The displayed "Scope" column shows nesting depth (0 = global, 1 = nested, ...). Sibling blocks at the same depth share the same number; correctness of lookup is unaffected because it relies on parent pointers, not the displayed number.
- A separate append-only log (SymLog) records every declaration ever made, including those from scopes that have already been exited. This log is used solely by the `print_symbol_table()` routine so that the report can show the complete history of declarations.
- Redeclaration is detected by searching only the current scope before inserting a new symbol.

10. Intermediate Code Generation (TAC)

The final required phase walks the validated AST and emits Three-Address Code. Each TAC instruction contains at most three operands (typically one operator and up to two source operands, with the result stored in a temporary or a named variable).

10.1 Constructs Handled

- Arithmetic expressions (respecting operator precedence via the AST)

- Relational and logical expressions
- Assignment statements
- Control flow (if, if-else, while) using labels and conditional/unconditional jumps (ifFalse ... goto Lx, goto Lx)
- print statements

10.2 Illustrative Example

Source fragment:

```
int a = 10;
int d = a / 2;
if (a != 0 && b > 10.0) {
    print a;
}
```

Generated TAC (excerpt):

```
t1 = 10
a = t1
t4 = 2
t5 = a / t4
d = t5
t8 = 0
t9 = a != t8
t10 = 10.00
t11 = b > t10
t12 = t9 && t11
ifFalse t12 goto L1
print a
L1:
```

Temporary names are generated sequentially (t1, t2, ...). Labels for control flow are likewise generated sequentially (L1, L2, ...). The generator never produces machine code or assembly; TAC is the final required output of the compiler.

11. Challenges

Dangling-else conflict. The classic shift/reduce conflict appeared as soon as if-else was added. It was resolved by accepting Bison's default shift behaviour, which correctly binds else to the nearest unmatched if.

Combined declaration + initialization. The manual's illustrative grammar shows separate declaration and assignment, yet the semantic-error examples use the combined form "bool b = 5 + 3.2;". Both forms were therefore supported by extending the decl_stmt production.

Two-pass lexical analysis. Flex's yylex is designed to be called on-demand by the parser. To give the lexical phase its own independent status report (as required by the pipeline diagram), a dedicated first pass that simply drains the token stream was introduced, followed by a rewind and a full parse.

Scope display versus live structure. Live scopes are freed on exit so that lookup remains correct. A separate persistent log was added solely for the printable symbol-table dump, ensuring closed-scope variables still appear in the report.

Multiple error reporting. Early prototypes stopped at the first semantic error. The final design accumulates all semantic errors and reports them together, matching the expectation that a single invalid program should surface every distinct violation.

12. Testing

A comprehensive test suite lives under `tests/` and `examples/`. Each category required by Section 15 of the Project Manual is covered.

12.1 Test Categories

Category	What is Demonstrated
Successful compilation	Non-trivial valid programs (<code>sample.mc</code> , scope tests) compile cleanly through to TAC
Lexical errors	Invalid characters (<code>\$ @ # ~</code>) reported with line numbers; compilation aborts
Syntax errors	Missing semicolons / malformed statements reported; error recovery continues
Semantic errors	All six required rules (undeclared, redeclaration, scope, type mismatch, invalid assignment, invalid expression) detected

12.2 Sample Output — Valid Program (Full Pipeline)

Command: `./build/compiler examples/sample.mc`

```
==== Lexical Analysis ====
Lexical Analysis: SUCCESS (no lexical errors)

==== Parsing ====
Parsing successful.

==== Abstract Syntax Tree ====
Program
  Decl (int a) [line 2]
    IntLit (10) [line 2]
  Decl (float b) [line 3]
    FloatLit (20.50) [line 3]
  Decl (bool c) [line 4]
    BoolLit (true) [line 4]
  Decl (int d) [line 7]
    BinOp (/) [line 7]
      Id (a) [line 7]
      IntLit (2) [line 7]
  Decl (int e) [line 8]
    BinOp (%) [line 8]
      Id (a) [line 8]
      IntLit (3) [line 8]
  If [line 14]
    Cond:
      BinOp (&&) [line 10]
```

Figure 12.2a: Valid run — Lexical Analysis, Parsing, and AST (part 1)

```

Cond:
  BinOp (&&) [line 10]
  BinOp (!=) [line 10]
    Id (a) [line 10]
    IntLit (0) [line 10]
  BinOp (>) [line 10]
    Id (b) [line 10]
    FloatLit (10.00) [line 10]
Then:
  Block
    Print [line 11]
    Id (a) [line 11]
While [line 17]
  Cond:
    UnOp (!) [line 14]
    BinOp (==) [line 14]
      Id (c) [line 14]
      BoolLit (false) [line 14]
  Body:
    Block
      Print [line 15]
      Id (b) [line 15]
      Assign (c) [line 16]
      BoolLit (false) [line 16]

```

Figure 12.2b: Valid run — AST continued (If / While nodes)

```

===== Semantic Analysis =====

===== Symbol Table =====
Name      Type      Scope      Line Declared
-----
a         int        0          2
b         float       0          3
c         bool        0          4
d         int         0          7
e         int         0          8

Semantic analysis successful. No errors found.

===== Three Address Code (TAC) =====
t1 = 10
a = t1
t2 = 20.50
b = t2
t3 = true
c = t3
t4 = 2
t5 = a / t4
d = t5
t6 = 3
t7 = a % t6

```

Figure 12.2c: Valid run — Semantic Analysis, Symbol Table, and TAC (part 1)

```

t7 = a % t6
e = t7
t8 = 0
t9 = a != t8
t10 = 10.00
t11 = b > t10
t12 = t9 && t11
ifFalse t12 goto L1
print a
L1:
L2:
t13 = false
t14 = c == t13
t15 = !t14
ifFalse t15 goto L3
print b
t16 = false
c = t16
goto L2
L3:

```

Figure 12.2d: Valid run — Three Address Code (continued)

12.3 Lexical Error Detection

Command: `./build/compiler tests/lexical_error.mc`

```
==== Lexical Analysis ====
Lexical Error at line 8: unexpected character '$'
Lexical Error at line 9: unexpected character '@'
Lexical Error at line 10: unexpected character '#'
Lexical Error at line 11: unexpected character '~'
Lexical Analysis: FAILED (4 lexical error(s) found)

Compilation failed: 4 lexical error(s).
```

Figure 12.3: Lexical error detection — unexpected characters reported with line numbers

12.4 Syntax Error Detection

Command: `./build/compiler tests/syntax_error.mc`

```
==== Parsing ====
Syntax Error at line 3: syntax error
Syntax Error at line 7: syntax error
Syntax Error at line 11: syntax error
Syntax Error at line 14: syntax error

Compilation failed: 4 syntax error(s).
```

Figure 12.4: Syntax error detection — multiple syntax errors with recovery

12.5 Semantic Error Detection

Command: `./build/compiler tests/semantic_error.mc` (or equivalent combined test)

```
==== Semantic Analysis ====
Semantic Error at line 2: redeclaration of variable 'x'
Semantic Error at line 5: undeclared variable 'undeclared_var'
Semantic Error at line 11: assignment to undeclared variable 'z'
Semantic Error at line 14: cannot assign 'float' expression to 'bool' variable 'flag'
Semantic Error at line 17: cannot assign 'bool' expression to 'int' variable 'a'
Semantic Error at line 20: logical operators require bool operands
```

Figure 12.5: Semantic error detection — redeclaration, undeclared, type mismatch, invalid expressions

12.6 Build and Execution Instructions

Build (from project root):

```
make
# produces ./build/compiler
```

Run:

```
./build/compiler <source-file>
./build/compiler <source-file> --phase=lexical|syntax|ast|semantic|symtab|tac
```

13. Team Contribution

GitHub activity is a required evaluation criterion. The repository shows regular, meaningful commits distributed across the development timeline (18 July – 29 July 2026). The table below summarises the primary contribution areas based on the commit history of the shared repository.

13.1 Contribution Summary by Area

Area / Module	Key Commits / Work	Primary Author(s)
Project setup & structure	Setup project structure, Add Makefile, root scaffolding	altafth / root
Lexer	Complete initial lexer setup; Update lexer with comment states and lexical error counting; Add tests for invalid operators	altafth, CodeByRafi
Parser & Grammar	Setup parser headers; Refactor parser.y to include ast.h; Document grammar rules, precedence and lexical tokens; Add formal CFG	altafth, CodeByRafi
AST	Add AST data structures and node construction utilities	altafth
Symbol Table	Implement basic scope management; Refactor semantic.h to symbol_table.h; Add scope test for variable visibility	CodeByRafi, altafth
Semantic Analyzer	Add semantic analysis interface; Add semantic analyzer with type and scope checking; Add semantic error tests	altafth, CodeByRafi
Code Generation (TAC)	Integrate code generation module into build system	altafth
Driver / Integration	Integrate compiler phases into main driver; Create initial module files	altafth
Documentation & Testing	Document architecture, sample outputs, testing summary, error catalog, design decisions; Refactor syntax_error.mc; fix sample.mc	altafth, CodeByRafi

13.2 Commit Timeline Highlights

Commits span from project scaffolding (18 July) through lexer, parser, AST, symbol table, semantic analysis, code generation, documentation, and final architecture/sample-output polish (29 July). Both primary GitHub accounts (altafth and CodeByRafi) contributed verified commits across multiple modules, demonstrating collaborative development rather than a single last-minute upload.

Date range	Focus of commits
18 Jul	Project structure, initial Makefile
19–21 Jul	Lexer setup, parser headers and global declarations
23–24 Jul	Lexer comment states & error counting; AST data structures
25 Jul	Makefile for Flex/Bison build
26–27 Jul	Parser–AST integration, sample program, semantic interface
28 Jul	Scope management, semantic analyzer, phase integration, lexical/syntax tests
29 Jul	Architecture docs, sample outputs, testing summary, CFG, error catalog, TAC integration

Note: The repository is maintained at <https://github.com/altafth/CC-Lab-Project-Group09>. Both team members (Altaf Hosen — altafth, and Rakibul Islam Rafi — CodeByRafi) contributed verified commits across multiple modules throughout the development timeline, demonstrating collaborative development. Each member is responsible for understanding the complete implementation as required by the AI Usage Policy and Individual Viva.

14. Conclusion

This project successfully integrated the six required modules of a compiler front-end into a single, coherent pipeline. The resulting system correctly tokenizes source programs, validates them against a formally defined CFG, constructs a hierarchical AST, maintains nested scopes, detects the full set of mandated semantic errors, and emits Three-Address Code for every supported language construct.

Working through the complete pipeline — rather than isolated lab exercises — provided a far deeper understanding of how the phases of a compiler communicate with and depend on one another. Particular attention was paid to clear diagnostic messages and to the ability of the compiler to reject invalid programs as reliably as it accepts valid ones.

The fixed language specification ensured a level playing field for evaluation while still leaving room for design decisions (combined declaration-initialization, two-pass lexical analysis, persistent symbol-table log, selective phase output). All mandatory requirements of Sections 4 and 5 of the Project Manual have been met by Team Binary Builders.

15. References

1. Compiler Construction Lab Project Manual, Department of CSE, Metropolitan University, Bangladesh (2026).
2. KhalidBinSelim, Compiler Construction Lab Project Template Repository, GitHub.
3. OpenAI ChatGPT, Anthropic Claude, and Alibaba Qwen (documentation assistance).
4. Compiler Construction Lab Course Lecture Notes.

Appendix A — Sample Program Source

examples/sample.mc (complete)

```
// This is a comment
int a = 10;
float b = 20.5;
bool c = true;

/* Block Comment Test */
int d = a / 2;      // Division test
int e = a % 3;      // Modulo test

if (a != 0 && b > 10.0) {
    print a;
}

while (!(c == false)) {
    print b;
    c = false;
}
```

Appendix B — Build System (Makefile)

Makefile

```
CC = gcc
CFLAGS = -Wall -g -Isrc/ast -Isrc/symbol_table -Isrc/semantic -Isrc/codegen -Ibuild
LEX = flex
YACC = bison
BUILD = build
SRC = src

all: $(BUILD)/compiler

$(BUILD):
    mkdir -p $(BUILD)

$(BUILD)/parser.tab.c $(BUILD)/parser.tab.h: $(SRC)/parser/parser.y | $(BUILD)
    $(YACC) -d -o $(BUILD)/parser.tab.c $(SRC)/parser/parser.y

$(BUILD)/lex.yy.c: $(SRC)/lexer/lexer.l $(BUILD)/parser.tab.h | $(BUILD)
    $(LEX) -o $(BUILD)/lex.yy.c $(SRC)/lexer/lexer.l

$(BUILD)/compiler: $(BUILD)/lex.yy.c $(BUILD)/parser.tab.c \
    $(SRC)/ast/ast.c \
    $(SRC)/symbol_table/symbol_table.c \
    $(SRC)/semantic/semantic.c \
    $(SRC)/codegen/codegen.c \
    $(SRC)/main.c
    $(CC) $(CFLAGS) -o $@ $^

clean:
    rm -rf $(BUILD)

.PHONY: all clean
```

— End of Project Report — Team Binary Builders —